Understanding graph representation learning in reinforcement learning based logic synthesis

Anonymous Author(s)
Affiliation
Address
email

Abstract

Computer chips ought to be as small as possible for cost efficiency. The problem of minimizing chip size while preserving the chip's functionality has been studied through the prism of and-inverter graphs (AIGs), since any Boolean function can be represented by an AIG. Because the complexity of AIG size minimization is unknown—it is an instance of the minimum circuit size problem, which lies in NP but is not known to be NP-hard—no procedure returns optimal AIGs at the sizes of interest, and there is therefore no supervision signal in the form of optimal AIGs to imitate. Reinforcement learning, and more generally planning, have been used to minimize AIGs in the absence of such a signal. Contemporarily, graph representation learning has been applied to train machine learning models to perform tasks on graph data. More recently, those two technologies have been combined and reported to outperform state-of-the-art logic synthesis heuristics, but the relative importance of graph representation learning in those methods remains unknown. Is graph representation learning useful when minimizing and-inverter graphs with reinforcement learning?

1 Introduction

A circuit that computes a given Boolean function with fewer logic gates occupies less area and is cheaper to manufacture [30]. Given a Boolean function, logic synthesis methods [7, 31, 23] are used to find a small circuit that computes it. Because every Boolean function can be represented with AND and NOT gates only, logic synthesis methods represent circuits with and-inverter graphs (AIGs) which nodes are AND gates and edges can invert incoming bits.

However, finding the smallest AIG encoding a given Boolean function is an open problem. It is an instance of the minimum circuit size problem, which lies in NP but is not known to be NP-hard [15], and which is related to Graph Isomorphism [2]. In practice, AIGs are reduced by applying sequences of heuristics that rewrite the graph while preserving functional equivalence. Many such heuristics are implemented in the `ABC` [5] software. Choosing the best heuristics sequence is a search problem.

In the absence of exact solvers, machine learning, such as supervised learning, has been applied to AIG minimization [38]. However it suffered from two major weaknesses: first, the supervision signal to train from could be a suboptimal AIG since no exact solver exists, and second, graphs generative models are still quite limited [38]. Successful machine learning approaches are unsupervised ones that only require a scalar signal, often the size of a given AIG which is cheap to obtain: Bayesian optimization [11], multi-armed bandits [29], reinforcement learning (RL) [14, 48] and Monte Carlo tree search [32]. Contemporarily, graph representation learning has been applied to train models on graph data, and circuits are graphs: task-agnostic encoders built specifically for AIGs are now available [18, 21, 46, 47]. The two lines of work have since been combined. Several of the methods above encode the current AIG with a graph neural network rather than with a fixed set of summary

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

statistics [48, 32, 3], and achieve reductions beyond the expert `ABC` heuristics. The motivation is the same as the one behind the very first deep RL algorithm, which was designed specifically for Backgammon [40]: the state space of AIG minimization is huge and cannot be enumerated, so the value of a state has to be generalized from other states. A learned representation of the circuit should carry structural information that statistics discard, and that is what should allow a deep reinforcement learning algorithm [36] to generalize to unseen circuits: "I have seen a similar graph before, I should probably edit the AIG in a similar way to get good results".

What those RL methods do not report is how much of their performance is attributable to the learned representation design. We are not aware of a study that isolates the contribution of the AIG encoder to RL training. Our work reports such an ablation, in the fixed-horizon setting that Zhu et al. [48] introduced and that later work reuses. We keep that pipeline fixed—the Markov decision process, the policy and value heads, the training algorithm and the hyper-parameter grid—and vary only how the AIG is presented to the policy. We show the following results.

Contributions: First, we show that adding a graph convolutional network to the policy does not improve AIG minimization beyond a multi-layer perceptron (MLP) that reads graph statistics (number of nodes, of levels, ...). Second, we give a candidate explanation: we compute, for AIG value prediction, the best error achievable by any model whose expressivity is bounded by the 1-dimensional Weisfeiler-Leman refinement—which covers message passing networks—and find it no better than the one achievable from the graph statistics alone. Next, we present existing AIG minimization methods.

2 Related work

2.1 Combinational optimization

Combinational synthesis is a class of fast heuristics that reduce an AIG by rewriting it locally, and that work well in practice [23, 24, 7]. `ABC` [5] is the reference implementation and is widely used both in industry and academia. Throughout this paper we call *primitives* its standard local transformations, `balance`, `rewrite`, `resub` and `refactor`, and *heuristics* the named sequences of primitives it ships, such as `resyn2`. Since no primitive is globally optimal, reduction is done by composing them into heuristics. Hand-designed heuristics are the reference points against which learned methods are measured: `resyn` [31] applies `balance`, `rewrite`, `balance`, `rewrite`, `balance`. Searching over sequences of primitives has been automated without learned circuit representations, with Bayesian optimization [11] and with multi-armed bandits [29].

2.2 Graph representation learning for AIGs

A separate line of work learns representations of circuits that are meant to transfer across circuits, rather than a policy for one circuit. DeepGate4 [47] is a foundation model that learns node embeddings of AIGs, supposedly useful for various downstream tasks on AIGs. Three encoders from this line—DeepGate [18], PolarGate [21] and FuncGNN [46]—are among the models we evaluate in section 6. `GraphBench` [38] is, to our knowledge, the first AIG reduction benchmark built for methods that must perform on unseen AIGs. It provides a training set of Boolean functions of varying input and output dimension and a disjoint test set, and it reports node reduction relative to the expert `ABC` heuristic: `resyn2`, `resyn2`, `dc2`, `resyn2rs`, `resyn2`, applied in that order. Its main result is negative: given the AIG obtained by the naive construction from a truth table, specialized DAG generative models [6, 19] do not preserve functional equivalence while reducing size.

2.3 Reinforcement learning for AIGs

The works discussed in this section use similar AIG minimization formulations: the state observed by the trained policy is some representation of the AIG to minimize and the actions are `ABC` primitives (section 2.1).

Hosny et al. [14] formulate synthesis as an MDP whose states are vectors of AIG statistics—numbers of primary inputs and outputs, nodes, edges, levels and latches, and the proportions of AND and NOT nodes—whose actions are `ABC` primitives, and whose reward combines node count and level count.

Zhu et al. [48] keep ABC primitives as actions but encode the whole AIG with a graph convolutional network, alongside a small vector of statistics and recent actions, and reward the relative reduction in node count. Episodes there have a fixed horizon and no early stopping, so the induced problem is a search over flows of fixed length. This is the formulation we adopt, horizon included, and we state it in full in section 4. Both Hosny et al. [14] and Zhu et al. [48] train with policy gradient methods [39, 44, 27, 36]. We are not aware of a value-based [43, 26] treatment, even though AIG editing is deterministic. These methods report minimization performance beyond the ABC heuristics, but they are not designed to generalize: a policy is trained to reduce one AIG, so the training cost is paid again for every new circuit. Search-based methods lift that restriction. AlphaSyn [32] and AlphaChip [22] plan online and therefore apply to any circuits: every AIG edit choice requires a lookahead, and every lookahead step requires network forward passes. However we argue that the cost of retraining a model is cheap compared to the cost of producing unnecessarily big chips at scale. ABC-RL [3] sits in between: a policy pre-trained on other circuits guides the tree search, and its influence is scaled down when the circuit at hand is far from anything in the training set. Two further formulations widen the setting rather than the state: ESE [34] adds technology mapping operations to the action space and trains with PPO [36], and CB-EVO [20] drops the sequential state altogether and treats the choice of primitive as a contextual bandit whose context is a set of circuit features.

2.4 Graph reinforcement learning for other combinatorial optimization

AIG reduction is an instance of graph structure optimization in the taxonomy of Darvariu et al. [8], who survey reinforcement learning applied to combinatorial problems on graphs and report that a graph neural network encoder is the common choice of state representation. Within chip design the same combination is used for transistor sizing [42] and for macro placement [22]; in both cases the encoder is motivated as the component that enables transfer to new circuits. Next, we formulate AIG minimization as a Markov decision problem.

3 Background material

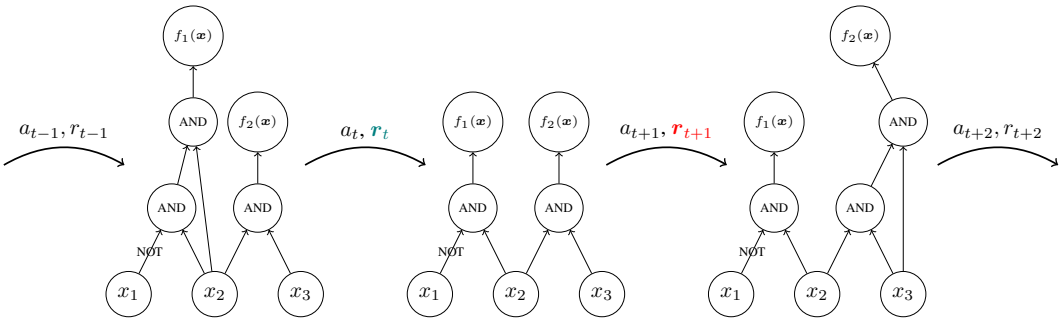

Figure 1: A trajectory in a generic MDP for AIG reduction, on AIGs of the Boolean function $f(x_1, x_2, x_3) = (\bar{x_1} \wedge x_2,\ x_2 \wedge x_3)$. Every method surveyed in section 2 acts at this granularity: each action is one ABC primitive applied to the whole graph, and the primitives of section 2.1 preserve functional equivalence by construction, so every state reachable from $\mathcal{G}_0$ still computes f.

3.1 And-inverter graphs minimization

Given an arbitrary Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, the objective is to find an and-inverter graph that encodes f with as few nodes as possible.

Section 3.2.1 from Stoll et al. [38] states that an and-inverter graph $\mathcal{G} := (V, E)$ is a directed acyclic graph of in-degree (fanin) at most two. The node set is partitioned into input nodes of in-degree 0, $V^{in} := \{v_1^{in}, \ldots, v_n^{in}\}$, AND nodes of in-degree two, $V^{AND} := \{v_1^{AND}, \ldots, v_k^{AND}\}$, and output nodes of in-degree one, $V^{out} := \{v_1^{out}, \ldots, v_m^{out}\}$; AIGs are therefore heterogeneous graphs [37]. The edge set is $E \subseteq V \times V \times \{0,1\}$, with 0 indicating direct transmission and 1 indicating input negation (NOT). For an input $\boldsymbol{x} \in \{0,1\}^n$, values are assigned to input nodes, propagated through gate nodes (applying inversion where specified), and collected at output nodes

to yield $\mathcal{G}(\boldsymbol{x}) \in \{0,1\}^m$. Because the combination of AND and NOT is functionally complete, any Boolean function can be represented using only these two operations.

We write $\mathcal{G}_f := \{\mathcal{G} : \mathcal{G}(\boldsymbol{x}) = f(\boldsymbol{x})\ \forall \boldsymbol{x} \in \{0,1\}^n\}$ for the set of AIGs functionally equivalent to f. AIGs are non-canonical representations of Boolean functions [25]—two different AIGs can represent the same function—so $|\mathcal{G}_f| > 1$ in general: e.g. the graphs of figure 1 are functionally equivalent. Given a Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, we want to find $\mathcal{G}^\star \subseteq \arg\min_{\mathcal{G} \in \mathcal{G}_f} |V|$, the AIGs functionally equivalent to f with as few nodes as possible. The hardness of this problem is not known: it is an instance of the Minimum Circuit Size Problem [15], a well-studied problem in NP that is not known to be NP-hard[1].

3.2 Markov decision process

AIG minimization can be formulated as a sequential decision making task in which the environment is the current AIG, the actions are edits of that AIG, and the rewards are shaped to favor edits that remove nodes. Formally, this is the problem of finding an optimal policy for a Markov decision process (MDP) [4, 33]. An MDP is a tuple $\mathcal{M} = \langle S, A, R, T, T_0 \rangle$. S is a finite set of states representing all possible configurations of the environment (e.g. AIGs, or AIG statistics). A is a finite set of actions available to the agent (e.g. AIG edits). $R : S \times A \to \mathbb{R}$ is a deterministic reward function that assigns a real-valued reward to each state-action pair (e.g. number of deleted nodes). $T : S \times A \to \Delta(S)$ is the transition function that maps state-action pairs to probability distributions over next states $\Delta(S)$ (e.g. how graph edits change the AIG; here the transitions are deterministic). $T_0 \in \Delta(S)$ is the initial distribution over states (e.g. a set of AIGs to reduce).

Given an MDP $\mathcal{M} \equiv \langle S, A, R, T, T_0 \rangle$, the goal of reinforcement learning is to find a policy $\pi : S \to A$ that maximizes the expected discounted sum of rewards $J(\pi) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 \sim T_0, s_{t+1} \sim T(s_t, \pi(s_t))\right]$ where $0 < \gamma \leq 1$ is the discount factor that controls the trade-off between immediate and future rewards.

We write $V^\star(s) = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(s_t, \pi(s_t)) \mid s_0 = s, s_{t+1} \sim T(s_t, \pi(s_t))\right]$ for the value of a state under an optimal policy, i.e. the discounted sum of rewards collected from s by acting optimally. $V^\star$ is the quantity a critic is trained to approximate in actor-critic methods, and the quantity a state representation must be able to distinguish states by; section 6 uses it to measure what a representation can express, independently of any training procedure.

3.3 Message passing neural networks

A message passing neural network (MPNN) [10] attaches a vector $h_v^{(0)}$ to each node v, initialized from that node's features, and refines it over k rounds: $h_v^{(t+1)} = \phi^{(t)}\left(h_v^{(t)}, \bigoplus_{u \in \mathcal{N}(v)} \psi^{(t)}\left(h_v^{(t)}, h_u^{(t)}, e_{uv}\right)\right)$, where $\mathcal{N}(v)$ are the neighbours of v, e_{uv} is an optional label on the edge between u and v, $\bigoplus$ is a permutation-invariant aggregator, and $\phi^{(t)}, \psi^{(t)}$ are learned. A graph-level vector is then obtained by pooling $\{h_v^{(k)}\}_{v \in V}$ with a second permutation-invariant readout. Architectures in this family differ only in the choice of ψ, $\bigoplus$ and ϕ: GCN [17], GIN [45], GAT [41] and GraphSAGE [13] are instances, and so are the AIG-specific encoders of appendix D. They also share a limitation. After k rounds, two nodes that k iterations of the 1-dimensional Weisfeiler-Leman refinement assign the same color carry the same embedding [45, 28, 1], so no network in the family can tell apart two AIGs that 1-WL cannot. Section 6.2 turns this into a lower bound on the error of every such encoder.

Next, we present our methodology for studying the effect of MPNNs when using RL to train policies for AIG minimization formulated as an MDP.

4 Methodology

We use the same methodology as Zhu et al. [48] who first combined graph convolutional networks and (deep) reinforcement learning for AIG minimization. We use a similar MDP to theirs: the actions are the five `ABC` primitives {`balance`, `rewrite`, `rewrite -z`, `refactor`, `refactor -z`}, transitions are deterministic and every primitive preserves functional equivalence, so every reachable state stays

[1] `https://mcsp.work/index.html`

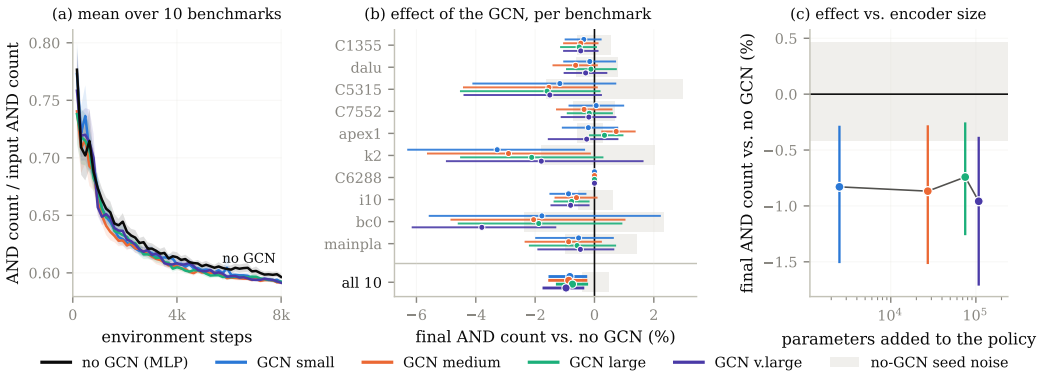

Figure 2: PPO on the ten benchmarks from Zhu et al. [48] with five observation encoders, each tuned (cf. appendix A.1 for more details). (a) AND count averaged over the ten benchmarks; (b) change in final AND count against the no-GCN policy, per benchmark and pooled over the ten (*all 10*), the grey band being the no-GCN policy's own spread over its five seeds; (c) the pooled change against the parameters the encoder adds to the policy. All intervals are 95% bootstrap confidence intervals over five seeds.

in $\mathcal{G}_f$, and an episode applies exactly $H = 20$ of them to the AIG $\mathcal{G}_0$ to reduce, which makes the induced optimization a search over flows of length 20 from a five-letter alphabet. The reward differs slightly: writing n_t for the number of AND nodes after t actions, $R(s_t, a_t) = 1 - n_{t+1}/n_0$ is issued at every step, so with $\gamma = 1$ we optimize node count alone—the objective of section 3.1—and not the multi-objective reward of Hosny et al. [14]. Unlike Zhu et al. [48], we use PPO [36] to train policies instead of REINFORCE [44] since the former is the more modern and state-of-the-art learner.

The policies we train always read a feature vector $x_{\mathcal{G}} \in \mathbb{R}^{|A|+5}$ holding the recent history of the actions taken so far, the AND and level counts of the current and previous AIG relative to $\mathcal{G}_0$, and the normalized time step: this is the same as in Hosny et al. [14], Zhu et al. [48]. This feature vector carries no AIG structural information, since two AIGs with the same size, depth and recent actions history are indistinguishable to it. The key novelty of Zhu et al. [48] is to feed the policy a graph convolutional network [17] reading the AIG and pooling it into a graph-level vector concatenated with the aforementioned feature vector $x_{\mathcal{G}}$. This GCN addition has since then been built upon (cf. section 2.3) but never ablated against e.g. Hosny et al. [14]. Ablating the GCN, which is the focus of the following sections, matters because while it may increase performance, it certainly adds memory and time overheads.

5 Graph reinforcement learning: reproducing Zhu et al. [48]

5.1 Setup

We minimize the same AIGs as Zhu et al. [48]: `apex1`, `bc0`, `C1355`, `C5315`, `C6288`, `C7552`, `dalu`, `i10`, `k2` and `mainpla`. They range from 504 to 6,359 AND gates and encode diverse Boolean functions. For each AIG we run PPO with five policies: no GCN, and a small, medium, large and very large GCN. Every (AIG, policy) pair is tuned over the same grid—three learning rates by three entropy coefficients, five seeds each—and we report the cell with the lowest mean final AND count over its five seeds. Each policy therefore receives the same tuning budget and is shown at its own optimum. We periodically save and evaluate the greedy PPO policy: we use it to minimize an AIG and report the AND count after applying $H = 20$ `ABC` primitives (cf. section 4). All hyper-parameters and implementation details are given in appendix A.1.

5.2 Results

Figure 2(a) shows that a tuned PPO learns to minimize an AIG independently of the policy architecture, reaching a median performance 6% below the `resyn2` logic synthesis heuristic [31]. Figure 4 in appendix A.2 breaks the same runs down by benchmark and the observation holds, and table 1 reports

AIG	Starting nodes	no GCN (MLP)	GCN small	GCN medium	GCN large	GCN v. large	`resyn2`
C1355	504	389 $\pm$3	387 $\pm$1	387 $\pm$0	387 $\pm$1	387 $\pm$0	390
dalu	1740	1008 $\pm$9	1006 $\pm$6	1001 $\pm$3	1006 $\pm$6	1005 $\pm$2	1052
C5315	1773	1303 $\pm$43	1287 $\pm$8	1283 $\pm$2	1282 $\pm$6	1283 $\pm$5	1309
C7552	2074	1371 $\pm$12	1372 $\pm$10	1367 $\pm$11	1369 $\pm$5	1369 $\pm$10	1455
apex1	2124	1066 $\pm$7	1064 $\pm$11	1074 $\pm$4	1070 $\pm$4	1063 $\pm$14	1179
k2	2190	1049 $\pm$26	1015 $\pm$32	1019 $\pm$27	1027 $\pm$21	1030 $\pm$36	1194
C6288	2337	1870 $\pm$0	1870 $\pm$0	1870 $\pm$0	1870 $\pm$0	1870 $\pm$0	1870
i10	2675	1715 $\pm$13	1700 $\pm$3	1705 $\pm$9	1702 $\pm$3	1702 $\pm$6	1829
bc0	3501	1512 $\pm$46	1485 $\pm$59	1481 $\pm$31	1484 $\pm$28	1455 $\pm$13	1713
mainpla	6359	2318 $\pm$37	2306 $\pm$14	2298 $\pm$11	2305 $\pm$23	2307 $\pm$14	2476

Table 1: Mean AND count after $H = 20$ `ABC` primitives, over the five seeds of each configuration's best hyper-parameter cell, $\pm$ one standard deviation across those seeds. *Starting nodes* is the AND count of the benchmark before any primitive is applied; `resyn2` is deterministic and has no spread. AIGs are ordered by input size.

the AND counts those runs reach. Pooled over the ten benchmarks, last row of figure 2(b), the GCN policies do end between 0.74% and 0.96% below the MLP policy with confidence intervals excluding zero, and the best GCN policies produce the smaller circuits on eight to nine benchmarks out of ten. The effect is nonetheless marginal: it is not larger than what re-running the baseline with different seeds produces (grey band), only six of the forty encoder-benchmark comparisons have a confidence interval entirely below zero, and in absolute terms the best GCN saves a median of 11 AND gates per benchmark, from none at all on `C6288` to 57 gates on `bc0`. Most importantly, figure 2(c) shows the gain does not increase with encoder size: $2{,}496$ added parameters improve performance as much as $107{,}968$. We repeated the experiment on the 100 AIGs (106 to $46{,}977$ AND gates) of the 2026 IWLS programming contest[2]. The picture there is the same in sign and magnitude, 0.3% to 0.7% below the MLP (appendix B).

Next, we ask why the gains from GCN policies are so marginal, despite the GCN being a representation specialized for graphs and the MLP reading only graph statistics and a short history of recent actions.

6 Why does graph representation learning improve so little over MLPs?

The previous section compares encoders inside one algorithm, so a null result there could be a property of that algorithm rather than of the representations. We therefore ask a question that does not involve a training algorithm at all: how well can the optimal value $V^\star$ of a state (cf. section 3.2) be predicted from what a given class of models is able to see? This kind of empirical analysis of graph representations, in particular of MPNNs (cf. section 3.3), has been carried out for graph-level regression tasks other than value prediction in Akbiyik et al. [1].

6.1 Building a dataset

Computing $V^\star$ exactly is out of reach: the tree of flows over the five primitives of section 4 has 5^H leaves, and every edge of it costs one `ABC` call on the full circuit. We therefore estimate it by sampling both the states and the continuations from them. A state is the AIG obtained by applying to the benchmark $\mathcal{G}_0$ a uniformly random flow of length $L \sim \mathcal{U}\{2, \dots, 10\}$; we draw $1{,}000$ such states per benchmark and store each one both as an `.aig` file and as the statistics $x_{\mathcal{G}}$ of section 4, so that every model class of section 6.2 is evaluated on exactly the same states and the same targets. From each state s we then run $M = 100$ independent rollouts of 8 further uniformly random primitives and keep the best AND count reached, $\widehat{V}(s) = \min_{j \le M} n^{(j)}(s)$, an upper bound on the minimum over the 5^8 flows of length eight that approaches it from above as M grows. A key difference to $V^\star$ should be kept in mind: $\widehat{V}$ truncates the horizon at eight steps instead of the $H = 20$ of the reinforcement learning experiments, so our targets approximate the value of an eight-step-to-go problem, and the bounds of section 6.2 are lower bounds on the error of predicting *that* quantity. The procedure is repeated with ten seeds per benchmark on the same ten AIGs as in section 5, and the figures of this

[2] `https://github.com/alanminko/iwls2026-ls-contest/`

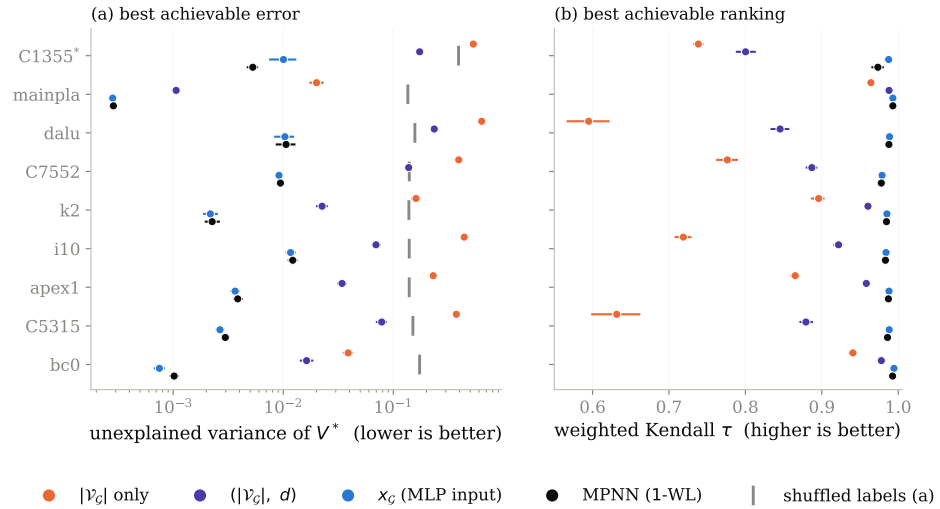

Figure 3: Best error (a) and best ranking (b) achievable by each model class on $\widehat{V}$, over nine benchmarks and ten dataset seeds. Each class is replaced by the optimal predictor for the information it can distinguish, so the value shown cannot be beaten by any model in that class. *`C1355` has only six distinct targets.

section report over those ten repetitions. The sampler is degenerate on two of them, reaching only two distinct values of $\widehat{V}$ on `C6288` and six on `C1355`: we exclude `C6288` and report over the remaining nine benchmarks, flagging `C1355` wherever it appears.

6.2 Theoretical lower bounds on the prediction error

A model that cannot distinguish two states must assign them the same prediction, so a class of models partitions the state space and its best achievable squared error is that of the predictor returning, for each part, the mean target over that part. No model in the class can beat that error, regardless of its size, its training procedure or its optimization. This is the protocol of `Graphtester` [1], which measures the theoretical boundary of MPNNs (cf. section 3.3) on a given dataset rather than the performance of one trained instance; we reimplement it for the target of section 6.1 and for four representations, ordered by what they can see: the number of nodes $|V_\mathcal{G}|$ alone, which is what a 0-layer message passing network reduces to; the pair $(|V_\mathcal{G}|, d_\mathcal{G})$, with $d_\mathcal{G}$ the logic depth reported by `ABC`; the engineered statistics $x_\mathcal{G}$ of section 4 in full, the action histogram included, which is exactly what the MLP baseline of section 4 reads; and the 1-WL coloring of the AIG, which upper bounds the separating power of every architecture of section 3.3. Comparing the last two is the question of this section: what a learned encoder could express, against what the statistics already express. They are incomparable rather than nested—only the current size and depth entries of $x_\mathcal{G}$ are functions of the current AIG, the previous step's size and depth, the normalized time step and the action histogram being episode history that no encoder of the current graph can recover—so the $x_\mathcal{G}$ bound may fall below the 1-WL bound without contradicting it, and a policy that reads both, as algorithm 1 does, is bounded by neither alone. Besides the squared error, which we normalize by the variance of the target, we report how well each class can *order* states, with Spearman's ρ, Kendall's τ and the weighted τ, since a critic that ranks successors correctly is useful even when its absolute errors are large. The bounds are computed in sample, so a partition fine enough to isolate every state would drive the error to zero regardless of whether the representation carries any signal about $\widehat{V}$; we control for this by recomputing every bound with the targets randomly permuted across states, which leaves the partition intact and destroys the association. Every bound we report sits 11 to 474 times below its shuffled-label counterpart, depending on the benchmark, and the two partitions we compare are of nearly equal granularity, at 865 parts for $x_\mathcal{G}$ and 860 for 1-WL out of 1,000 states on average, so neither class is merely memorizing the targets nor given more degrees of freedom than the other. Appendix C.1 details how the partitions are computed.

6.3 Results

A k-layer MPNN separates graphs at most as finely as k iterations of 1-WL, so grouping states by their 1-WL color and predicting each group's mean $\widehat{V}$ gives an error no MPNN can beat; the same construction bounds a 0-layer network, a $(\text{size}, \text{depth})$ model, and a model seeing the MLP's features $x_{\mathcal{G}}$. From figure 3, we observe that message passing is worth a great deal over global descriptors: the bound falls from 0.02–0.64 of the target variance for node count alone, to 1.1×10^{-3}–0.24 once depth is added, and to 3×10^{-4}–1.2×10^{-2} for 1-WL. It is worth nothing over $x_{\mathcal{G}}$. On eight of the nine benchmarks the $x_{\mathcal{G}}$ bound is the *lower* of the two, at 0.73 to 0.99 times the MPNN bound; only `C1355` goes the other way, where message passing is better by $1.93\times$. The ordering is stable within a dataset: paired by seed, $x_{\mathcal{G}}$ is lower on all ten seeds of six benchmarks and on seven and eight of ten for `dalu` and `C7552`. For ranking the two classes are indistinguishable—weighted Kendall $\tau \geq 0.97$ for both on every benchmark, with $x_{\mathcal{G}}$ ahead by at most 0.014—while size and depth alone reach only 0.60–0.99. Depth is irrelevant—the bound moves by at most 1.1×10^{-3} between $k = 1$ and $k = 50$, and edge direction and inverter bits change it by less than 10^{-6} (appendix C.2). `C6288` is excluded: its target takes only two distinct values.

Limitations of the bounds. Two restrictions should be read with the numbers above. First, they hold over the state distribution of section 6.1—AIGs reached by uniformly random flows of length two to ten—and not over the states a trained policy visits, which are fewer and more correlated; a representation could be uninformative on the former and useful on the latter. Second, the 1-WL argument covers message passing networks whose input is the AIG with node-type features, which is the setting of section 4 and of Zhu et al. [48]. It does not cover higher-order architectures, models given node identifiers or positional encodings, nor encoders whose node features are not a function of the graph alone—the simulation-derived features used by some AIG-specific models are an example, and for those the bound of figure 3 is not binding.

7 Discussion

In section 5, we observed that training a graph representation in the form of a GCN policy does not improve RL-based AIG minimization over an MLP reading graph statistics and the last few actions. In section 6.2, we approximated an upper bound on the performance of any MPNN predicting the state values $\widehat{V}$ of section 6.1. Those bounds give a candidate explanation for that null result: for predicting $\widehat{V}$, the statistics $x_{\mathcal{G}}$ the MLP reads are as expressive as any MPNN on the states we sample. On eight of the nine benchmarks the error achievable from $x_{\mathcal{G}}$ is the lower of the two, and for ranking states the two classes are indistinguishable, at weighted Kendall $\tau \geq 0.97$ for both. Nothing a GCN could extract from the graph would then improve the value estimate the policy is trained against, and the reduction it buys would stay of the order we measure. The bounds do show that message passing separates states far better than size and depth alone, so this is a statement about the specific comparison with $x_{\mathcal{G}}$, not about message passing being uninformative.

The practical consequence is that a graph encoder should not be assumed to pay for its parameters here: in this pipeline it buys under 1% of AND count, and the statistics baseline should be run before it is adopted. This is not a statement about graph representation learning for logic synthesis in general: our evidence covers one policy gradient algorithm, five `ABC` primitives and a fixed horizon. Appendix D reports a preliminary attempt to train actual graph models and AIG-specific models on these datasets; the models do not fit their training set, so we draw no conclusion from it.

8 Future work

Our results are restricted to the setting used: a policy gradient algorithm, five `ABC` primitives as actions, and the fixed horizon of section 4. Two of those choices—the algorithm and the action space—bear directly on how much a learned representation of the circuit could contribute, and relaxing them is where we would look next.

Other reinforcement learning algorithms. AIG editing is deterministic and its exploration problem is hard, a combination for which value-based algorithms [43, 26] are usually competitive, yet the

existing work on AIG reduction surveyed in section 2 is entirely policy gradient-based. That distinction matters here rather than only for sample efficiency: a value-based agent acts by comparing the predicted values of successor states, so its behavior rests entirely on the quantity section 6 measures, whereas in an actor-critic method the value estimate only shapes the advantage. If the representation is the bottleneck anywhere, it should be there, and our experiments do not answer whether a graph encoder helps under such an algorithm.

The full `ABC` action space. Five primitives give the policy little to decide, and the states it can reach are the images of a handful of short sequences, which may be why the statistics of section 4 suffice to tell them apart. `ABC` exposes many more primitives. With more primitives, reachable states might differ from one another in more ways, and a representation that sees the circuit rather than its summary would have correspondingly more to distinguish. Rerunning both the ablation of section 5 and the bounds of section 6.2 over that action space would say whether our negative result is a property of AIG reduction itself or of the small action space it is usually given.

To conclude, we have proposed the first empirical study of graph representation learning in RL-based logic synthesis. We hope our work will help researchers design better RL algorithms for AIG minimization by choosing the right representations that help policies accurately distinguish valuable AIG states.

References

[1] Eren Akbiyik, Florian Grötschla, Béni Egressy, and Roger Wattenhofer. Graphtester: Exploring Theoretical Boundaries of GNNs on Graph Datasets. In *Data-centric Machine Learning Research (DMLR) Workshop at ICML 2023, Honolulu, Hawaii*, July 2023.

[2] Eric Allender and Bireswar Das. Zero knowledge and circuit minimization. *Information and Computation*, 256:2–8, 2017. ISSN 0890-5401. doi: https://doi.org/10.1016/j.ic.2017.04.004. URL `https://www.sciencedirect.com/science/article/pii/S0890540117300512`.

[3] Animesh Basak Chowdhury, Marco Romanelli, Benjamin Tan, Ramesh Karri, and Siddharth Garg. Retrieval-guided reinforcement learning for boolean circuit minimization. In B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun, editors, *International Conference on Learning Representations*, volume 2024, pages 35039–35060, 2024. URL `https://proceedings.iclr.cc/paper_files/paper/2024/file/96bbdd0ed2a9e7cd2fb7caf2fae15f3d-Paper-Conference.pdf`.

[4] Richard Bellman. *Dynamic Programming*. 1957.

[5] Robert K. Brayton and Alan Mishchenko. Abc: An academic industrial-strength verification tool. In *International Conference on Computer Aided Verification*, 2010. URL `https://api.semanticscholar.org/CorpusID:1251520`.

[6] Alba Carballo-Castro, Manuel Madeira, Yiming QIN, Dorina Thanou, and Pascal Frossard. Generating directed graphs with dual attention and asymmetric encoding. In *The Fourteenth International Conference on Learning Representations*, 2026. URL `https://openreview.net/forum?id=s2s5xGQCKM`.

[7] Andrea Costamagna, Alan Mishchenko, Satrajit Chatterjee, and Giovanni De Micheli. An enhanced resubstitution algorithm for area-oriented logic optimization. In *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pages 1–5, 2024. doi: 10.1109/ISCAS58744.2024.10558264.

[8] Victor-Alexandru Darvariu, Stephen Hailes, and Mirco Musolesi. Graph reinforcement learning for combinatorial optimization: A survey and unifying perspective. *Transactions on Machine Learning Research*, 2024. ISSN 2835-8856. URL `https://openreview.net/forum?id=HduK51xNtS`. Survey Certification.

[9] Matthias Fey and Jan Eric Lenssen. Fast graph representation learning with pytorch geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[10] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 1263–1272. PMLR, 2017.

[11] Antoine Grosnit, Cedric Malherbe, Rasul Tutunov, Xingchen Wan, Jun Wang, and Haitham Bou Ammar. Boils: Bayesian optimisation for logic synthesis. In *Design, Automation, Test in Europe Conference, 2022*, pages 1193–1196, 2022. doi: 10.23919/DATE54114.2022.9774632.

[12] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using networkx. In *Proceedings of the 7th Python in Science Conference*, pages 11–15, 2008.

[13] William L. Hamilton, Rex Ying, and Jure Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[14] Abdelrahman Hosny, Soheil Hashemi, Mohamed Shalan, and Sherief Reda. Drills: Deep reinforcement learning for logic synthesis. In *2020 25th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pages 581–586, 2020. doi: 10.1109/ASP-DAC47756.2020.9045559.

[15] Valentine Kabanets and Jin-Yi Cai. Circuit minimization problem. In *Proceedings of the thirty-second annual ACM symposium on Theory of computing*, pages 73–79, 2000.

[16] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015.

[17] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. In *International Conference on Learning Representations*, 2017.

[18] Min Li, Sadaf Khan, Zhengyuan Shi, Naixing Wang, Huang Yu, and Qiang Xu. Deepgate: Learning neural representations of logic gates. In *Proceedings of the 59th ACM/IEEE Design Automation Conference*, pages 667–672, 2022.

[19] Mufei Li, Viraj Shitole, Eli Chien, Changhai Man, Zhaodong Wang, Srinivas, Ying Zhang, Tushar Krishna, and Pan Li. LayerDAG: A layerwise autoregressive diffusion model of directed acyclic graphs for system. In *Machine Learning for Computer Architecture and Systems 2024*, 2024. URL `https://openreview.net/forum?id=IsarrieeQA`.

[20] Fangzhou Liu, Wuqian Tang, Zehua Pei, Ziyang Yu, Haisheng Zheng, Zhuolun He, Mengjia Dai, Tinghuan Chen, and Bei Yu. Cb-evo: Contextual bandit tuning with evolutionary search for logic synthesis. *ACM Trans. Des. Autom. Electron. Syst.*, 31(6), June 2026. ISSN 1084-4309. doi: 10.1145/3779431. URL `https://doi.org/10.1145/3779431`.

[21] Jiawei Liu, Jianwang Zhai, Mingyu Zhao, Zhe Lin, Bei Yu, and Chuan Shi. Polargate: Breaking the functionality representation bottleneck of and-inverter graph neural network. In *Proceedings of the 43rd IEEE/ACM International Conference on Computer-Aided Design*, 2024.

[22] Azalia Mirhoseini, Anna Goldie, Mustafa Yazgan, Joe Wenjie Jiang, Ebrahim M. Songhori, Shen Wang, Young-Joon Lee, Eric Johnson, Omkar Pathak, Azade Nazi, Jiwoo Pak, Andy Tong, Kavya Srinivasa, Will Hang, Emre Tuncer, Quoc V. Le, James Laudon, Richard Ho, Roger Carpenter, and Jeff Dean. A graph placement methodology for fast chip design. *Nature*, 594: 207 – 212, 2021. URL `https://api.semanticscholar.org/CorpusID:235395490`.

[23] A. Mishchenko, S. Chatterjee, and R. Brayton. Dag-aware aig rewriting: a fresh look at combinational logic synthesis. In *2006 43rd ACM/IEEE Design Automation Conference*, pages 532–535, 2006. doi: 10.1145/1146909.1147048.

[24] Alan Mishchenko and Robert K. Brayton. Scalable logic synthesis using a simple circuit structure. 2006. URL `https://api.semanticscholar.org/CorpusID:8597391`.

[25] Alan Mishchenko, Satrajit Chatterjee, Roland Jiang, and Robert K. Brayton. Fraigs: A unifying representation for logic synthesis and verification, 2005. URL `https://api.semanticscholar.org/CorpusID:9209313`.

[26] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.

[27] Volodymyr Mnih, Adria Puigdomenech Badia, Mehdi Mirza, Alex Graves, Timothy Lillicrap, Tim Harley, David Silver, and Koray Kavukcuoglu. Asynchronous methods for deep reinforcement learning. In Maria Florina Balcan and Kilian Q. Weinberger, editors, *Proceedings of The 33rd International Conference on Machine Learning*, volume 48 of *Proceedings of Machine Learning Research*, pages 1928–1937, New York, New York, USA, 20–22 Jun 2016. PMLR. URL https://proceedings.mlr.press/v48/mniha16.html.

[28] Christopher Morris, Martin Ritzert, Matthias Fey, William L. Hamilton, Jan Eric Lenssen, Gaurav Rattan, and Martin Grohe. Weisfeiler and leman go neural: Higher-order graph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 4602–4609, 2019.

[29] Walter Lau Neto, Yingjie Li, Pierre-Emmanuel Gaillardon, and Cunxi Yu. Flowtune: End-to-end automatic logic optimization exploration via domain-specific multiarmed bandit. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 42(6):1912–1925, 2023. doi: 10.1109/TCAD.2022.3213611.

[30] Suhua Ou, Qingshan Yang, and Jian Liu. The global production pattern of the semiconductor industry: an empirical research based on trade network. *Humanities and Social Sciences Communications*, 11(1):1–13, 2024.

[31] Rajendran Panda and Farid N. Najm. Post-mapping transformations for low-power synthesis. *VLSI Design*, 7:289–301, 1998. URL https://api.semanticscholar.org/CorpusID:6811282.

[32] Zehua Pei, Fangzhou Liu, Zhuolun He, Guojin Chen, Haisheng Zheng, Keren Zhu, and Bei Yu. Alphasyn: Logic synthesis optimization with efficient monte carlo tree search. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9, 2023. doi: 10.1109/ICCAD57390.2023.10323856.

[33] Martin L. Puterman. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. John Wiley & Sons, 1994.

[34] Yu Qian, Xuegong Zhou, Hao Zhou, and Lingli Wang. An efficient reinforcement learning based framework for exploring logic synthesis. *ACM Trans. Des. Autom. Electron. Syst.*, 29(2), January 2024. ISSN 1084-4309. doi: 10.1145/3632174. URL https://doi.org/10.1145/3632174.

[35] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, and Noah Dormann. Stable-baselines3: Reliable reinforcement learning implementations. *Journal of Machine Learning Research*, 22(268):1–8, 2021.

[36] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.

[37] Chuan Shi, Yitong Li, Jiawei Zhang, Yizhou Sun, and Philip S Yu. A survey of heterogeneous information network analysis. *IEEE Transactions on Knowledge and Data Engineering*, 29(1): 17–37, 2016.

[38] Timo Stoll, Chendi Qian, Ben Finkelshtein, Ali Parviz, Darius Weber, Fabrizio Frasca, Hadar Shavit, Antoine Siraudin, Arman Mielke, Marie Anastacio, Erik Müller, Maya Bechler-Speicher, Michael Bronstein, Mikhail Galkin, Holger Hoos, Mathias Niepert, Bryan Perozzi, Jan Tönshoff, and Christopher Morris. Graphbench: Next-generation graph learning benchmarking, 2026. URL https://arxiv.org/abs/2512.04475.

[39] Richard S Sutton, David McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In S. Solla, T. Leen, and K. Müller, editors, *Advances in Neural Information Processing Systems*, volume 12. MIT Press, 1999. URL https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf.

[40] Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, March 1995. ISSN 0001-0782. doi: 10.1145/203330.203343. URL https://doi.org/10.1145/203330.203343.

[41] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. Graph attention networks. In *International Conference on Learning Representations*, 2018.

[42] Hanrui Wang, Kuan Wang, Jiacheng Yang, Linxiao Shen, Nan Sun, Hae-Seung Lee, and Song Han. Gcn-rl circuit designer: Transferable transistor sizing with graph neural networks and reinforcement learning. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2020. doi: 10.1109/DAC18072.2020.9218757.

[43] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.

[44] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Mach. Learn.*, 8(3–4):229–256, May 1992. ISSN 0885-6125. doi: 10.1007/BF00992696. URL https://doi.org/10.1007/BF00992696.

[45] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[46] Qiyun Zhao. Funcgnn: Learning functional semantics of logic circuits with graph neural networks, 2025.

[47] Ziyang Zheng, Shan Huang, Jianyuan Zhong, Zhengyuan Shi, Guohao Dai, Ningyi Xu, and Qiang Xu. Deepgate4: Efficient and effective representation learning for circuit design at scale. In *The Thirteenth International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=b10lRabU9W.

[48] Keren Zhu, Mingjie Liu, Hao Chen, Zheng Zhao, and David Z. Pan. Exploring logic optimizations with reinforcement learning and graph convolutional network. In *2020 ACM/IEEE 2nd Workshop on Machine Learning for CAD (MLCAD)*, pages 145–150, 2020. doi: 10.1145/3380446.3430622.

A Additional details for graph reinforcement learning

A.1 Graph encoders and reinforcement learning

The environment. The MDP of section 4 instantiates the MDP of section 3.2 with the five `ABC` primitives as actions, where `-z` enables zero-cost replacements. Episodes start from $\mathcal{G}_0$ and last exactly $H = 20$ actions with no early stopping. Because every primitive preserves functional equivalence, the problem of section 3.1 is never violated, only under-solved. The reward is a level rather than a difference, so the return is maximized by an agent that keeps the AIG small throughout the episode and not merely at the end. Our environment also implements a seven-action variant adding `resub` and `resub -z`, and the full `ABC` action space, neither of which is used in our experiments.

The observation. Writing n_t and ℓ_t for the number of AND nodes and of levels after t actions, the vector part of the observation is

$$x_{\mathcal{G}} = \Big[\underbrace{\tfrac{1}{q} \textstyle\sum_{j=1}^{q} \mathbf{1}[a_{t-j} = a]}_{a \in A}, \;\; \frac{n_t}{n_0}, \; \frac{\ell_t}{\ell_0}, \; \frac{n_{t-1}}{n_0}, \; \frac{\ell_{t-1}}{\ell_0}, \; \frac{t}{H} \Big].$$

The first $|A| = 5$ entries are a normalized histogram of the last q actions, with $q = 3$ like in Zhu et al. [48]; the next four are the AND and level counts of the current and of the previous AIG, each divided by its value in $\mathcal{G}_0$ so that the vector is scale-free across benchmarks of very different sizes; the last is the normalized time step, which makes the finite-horizon problem Markov in the observation. It is computed in a single pass over the graph and adds no parameters of its own. The graph part of the observation is the AIG itself, as in Zhu et al. [48]: nodes carry a one-hot encoding of their type among the six types `abc_py` exposes, a directed edge runs from each fanin to the node it feeds, and edges carry no label.

The encoders. Table 2 lists the four graph encoders of section 4: three at depth three with widths from 12 to 128, and one at depth five, so that depth and width are not confounded. Each runs the message passing rounds of section 3.3 over the AIG and pools the node embeddings into a graph-level vector concatenated with $x_{\mathcal{G}}$ before the policy and value heads. The baseline uses no encoder at all and reads $x_{\mathcal{G}}$ alone through a multi-layer perceptron.

Table 2: The four graph encoders. Depth is the number of message passing rounds, width the hidden dimension, and output the dimension of the pooled vector concatenated with $x_{\mathcal{G}}$.

encoder	depth	width	output
`small`	3	12	4
`medium`	3	64	32
`large`	3	128	64
`v. large`	5	128	64

Training. Policies are trained with the PPO implementation of `stable-baselines3` [35] with $\gamma = 1$, the horizon being finite and fixed, and the encoder is trained end to end with the policy and value heads rather than pre-trained or frozen. Eight environments run in parallel and each contributes 20 steps per update, so an update is computed from 160 transitions and a single episode spans five updates; 8,000 environment steps therefore amount to $N = 50$ updates. After every update we evaluate the greedy policy for one episode, which is the quantity the learning curves report. The policy and value heads are each two hidden layers of 256 units and share the encoder of line 7 of algorithm 1. Algorithm 1 states the resulting procedure, which we call GRAPHPPO: it is ordinary PPO with the encoder made explicit, the encoding of the current AIG being the only place where the five configurations we compare differ.

The grid of section 5 crosses three learning rates (10^{-4}, 10^{-5}, 10^{-6}), three entropy coefficients (0, 0.01, 0.1) and five seeds, which with five encoders and ten benchmarks gives the 2,250 runs reported there. Twenty-six of those runs (eighteen `small`, eight `v. large`) stopped before the fiftieth update; we drop them rather than pad them, and requiring all five encoders to have completed a given (benchmark, learning rate, entropy, seed) cell leaves 424 of the 450 cells.

Algorithm 1 GRAPHPPO. The encoder f_θ is the only component that varies across the runs we compare; everything else is held fixed.

Require: AIG $\mathcal{G}_0$ to reduce with n_0 AND nodes, encoder f_θ, policy head π_θ, value head V_θ, action set A of five `ABC` primitives, fixed horizon $H = 20$, iterations N, epochs K, clipping ϵ, entropy weight β, value weight c, step size η

$n^\star \leftarrow n_0$
for $i = 1, \dots, N$ **do**
  $\mathcal{D} \leftarrow \emptyset$
  for each parallel environment **do**
    $\mathcal{G} \leftarrow \mathcal{G}_0$
    for $t = 0, \dots, H-1$ **do**
      $s_t \leftarrow \big(f_\theta(\mathcal{G}),\ x_{\mathcal{G}}\big)$ ▷ f_θ is a GCN, or is absent for the baseline
      $a_t \sim \pi_\theta(\cdot \mid s_t), \quad a_t \in A$
      $\mathcal{G} \leftarrow \textsc{Abc}(\mathcal{G}, a_t)$ ▷ deterministic, preserves functional equivalence
      $r_t \leftarrow 1 - |V^{AND}(\mathcal{G})| \,/\, n_0$ ▷ relative size, issued at every step
      $\mathcal{D} \leftarrow \mathcal{D} \cup \{(s_t, a_t, r_t, \log \pi_\theta(a_t \mid s_t), V_\theta(s_t))\}$
      $n^\star \leftarrow \min\big(n^\star,\ |V^{AND}(\mathcal{G})|\big)$
    end for
  end for
  compute returns $\hat{R}_t$ and advantages $\hat{A}_t$ from $\mathcal{D}$
  $\theta_{\text{old}} \leftarrow \theta$
  for K epochs over minibatches of $\mathcal{D}$ **do**
    $\rho_t(\theta) \leftarrow \pi_\theta(a_t \mid s_t) \,/\, \pi_{\theta_{\text{old}}}(a_t \mid s_t)$
    $L(\theta) \leftarrow \widehat{\mathbb{E}}_t\big[\min\big(\rho_t \hat{A}_t,\ \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t\big) - c\big(V_\theta(s_t) - \hat{R}_t\big)^2 + \beta\, \mathcal{H}\big[\pi_\theta(\cdot \mid s_t)\big]\big]$
    $\theta \leftarrow \theta + \eta \nabla_\theta L(\theta)$ ▷ gradients flow through f_θ
  end for
end for
return $n^\star$, the smallest AND count visited

A.2 Per-benchmark learning curves

Figure 4 resolves the aggregate of figure 2(a) into one learning curve per benchmark.

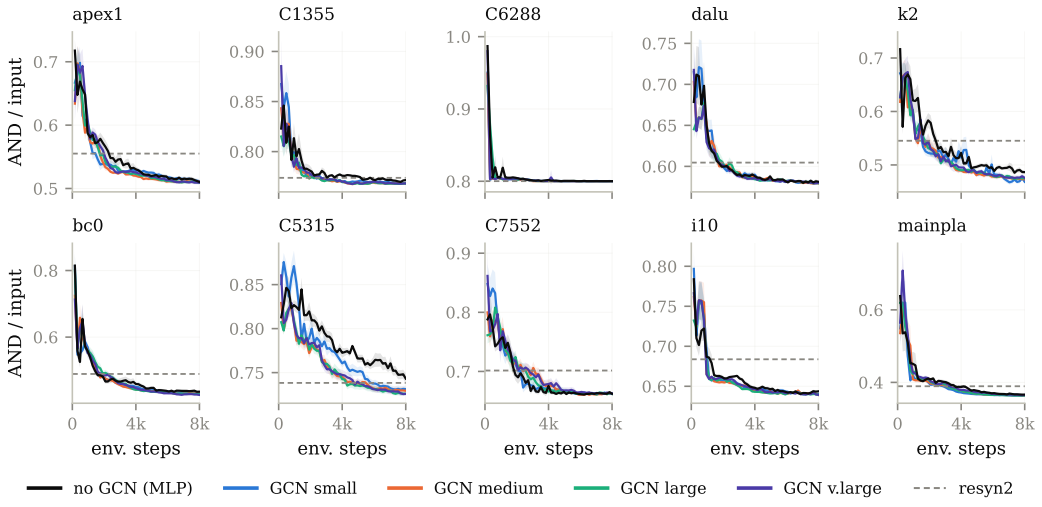

Figure 4: Per-benchmark learning curves for the runs aggregated in figure 2(a). Dashed line: `resyn2`.

B The IWLS programming contest

We repeat the comparison of section 5 on `ex200`–`ex299`, a suite spanning 106 to 46,977 input AND gates and therefore both larger and more varied than the ten MLCAD circuits. Each encoder ran at its own baseline hyper-parameters here instead of over the matched grid. The three GCNs reduce the final AND count by 0.33–0.71% against a reseed standard deviation of 0.54% (figure 5), and the difference is flat across two and a half orders of magnitude of circuit size (figure 6), so the conclusion of section 5 appears to hold here as well. The MLP runs were executed on CPU and did not finish on the largest circuits, which is why 20 of the 100 benchmarks are excluded; the comparison is therefore made on the smaller eighty, and we cannot rule out that the largest circuits behave differently.

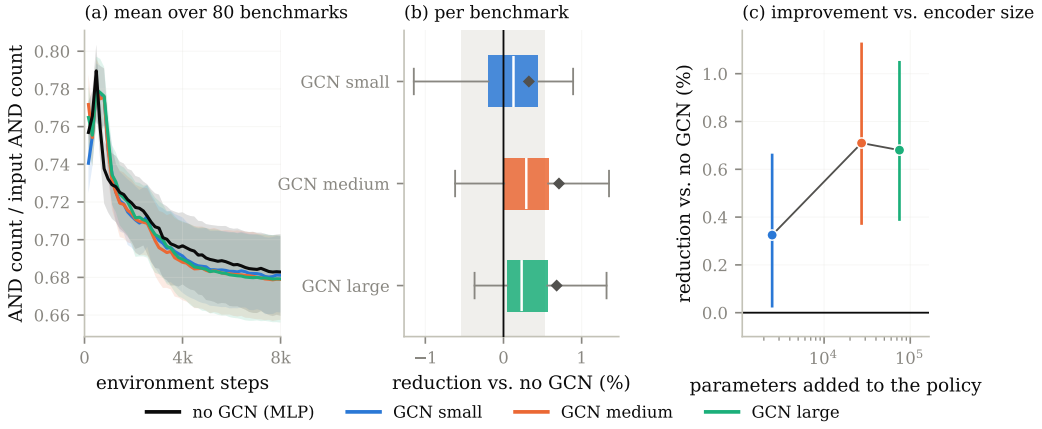

Figure 5: The comparison of figure 2 on `ex200`–`ex299`, where each encoder ran at its own baseline hyper-parameters instead of the matched grid. Restricted to the 390 of 500 (benchmark, seed) cells all four encoders completed, covering 80 benchmarks.

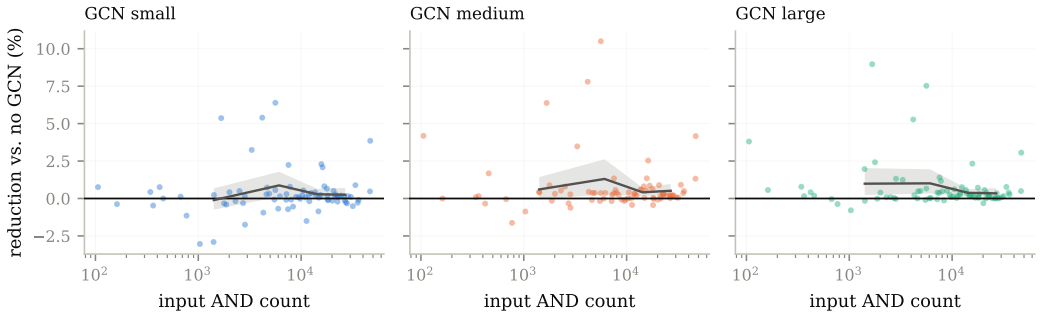

Figure 6: Reduction against the MLP as a function of input circuit size on `ex200`–`ex299`. Line: mean over size quartiles.

C Value prediction details

The datasets. The states of section 6.1 are drawn by sampling L primitives independently and uniformly from the five-element action set and running them in order; the graph models read the stored `.aig` file and the MLP reads the statistics $x_{\mathcal{G}}$ recorded at that point of the flow. The state is reloaded between the M rollouts. Producing one dataset costs $1{,}000 \times (L + M \times 8) \approx 8 \times 10^5$ `ABC` primitive calls per benchmark and per seed, which is why M is lowered to 10 on `C6288`, by far the most expensive circuit of the suite to rewrite. Each of the ten seeds (0 to 9) draws its own $1{,}000$ states and its own rollouts, and each resulting dataset is split at random into training, validation and test sets. That the target is expressed in AND nodes rather than in returns does not affect the comparison between model classes: the map $n \mapsto 1 - n/n_0$ from AND count to the reward of section 4 is affine and order-reversing, all the errors we report are ratios to the variance of the target and are therefore free of its units, and the rank correlations we report are unchanged up to sign.

Training. The models of appendix D are trained in mini-batches of 16 unless the batch size is itself searched over. Each architecture's grid of 36 configurations crosses three learning rates (10^{-3}, 5×10^{-4}, 10^{-4}) with a width and a depth, or with the number of attention heads for GAT and the polarity weight for PolarGate. DeepGate is the exception, with 12 configurations: its pretrained encoder is frozen, so only the two-layer head is tuned.

C.1 Computing the bounds

For the size and the size-and-depth classes of section 6.2 the partition is by exact equality of the descriptors. For $x_{\mathcal{G}}$, the full feature vector is snapped to a grid of spacing 10^{-8} and states are grouped by equality of the snapped vector, so the model we bound and the MLP baseline of section 4 see the same thing. For message passing, we group states by their 1-WL graph hash, computed with `networkx`'s [12] `weisfeiler_lehman_graph_hash` at every number of iterations $k = 1, \dots, 50$, with each node labeled by its type.[3] We bound four views of the AIG, which is what the four message passing curves of figure 3 report: the undirected graph $\mathcal{G}$, the directed graph $\mathcal{G}^{\rightarrow}$ in which refinement follows the fanin relation, and the two variants $\mathcal{G}^{e}, \mathcal{G}^{\rightarrow e}$ that additionally label each edge with its complement bit, so that inversion is visible to the refinement rather than only through node types. Errors are divided by $\max(1, \operatorname{Var} \widehat{V})$; in AND-node units the variance is orders of magnitude above one on every benchmark, so the clamp never binds and the reported quantity is the plain normalized error. Rank correlations are computed at $k = 50$ refinement iterations. The permutation control uses 25 permutations per dataset, 8 on `mainpla` and `C6288`, and is applied to the rank correlations as well, where the null is the correlation between $\widehat{V}$ and a random permutation of itself.

C.2 The message passing bound against depth

Figure 7 plots the message passing bound of section 6.2 against the number of refinement iterations k, for the four views of the AIG bounded there. The bound moves by at most 1.1×10^{-3} between $k = 1$ and $k = 50$, and labeling edges with direction or with the complement bit changes it by less than 10^{-6}, so neither deeper refinement nor a richer view of the graph brings a message passing network closer to $\widehat{V}$.

[3] The hash is a 16-byte `blake2b` digest. A collision would merge two groups that 1-WL separates and can only raise the reported value, so the bound would become conservative rather than optimistic; at $1{,}000$ graphs per dataset the probability of one is negligible.

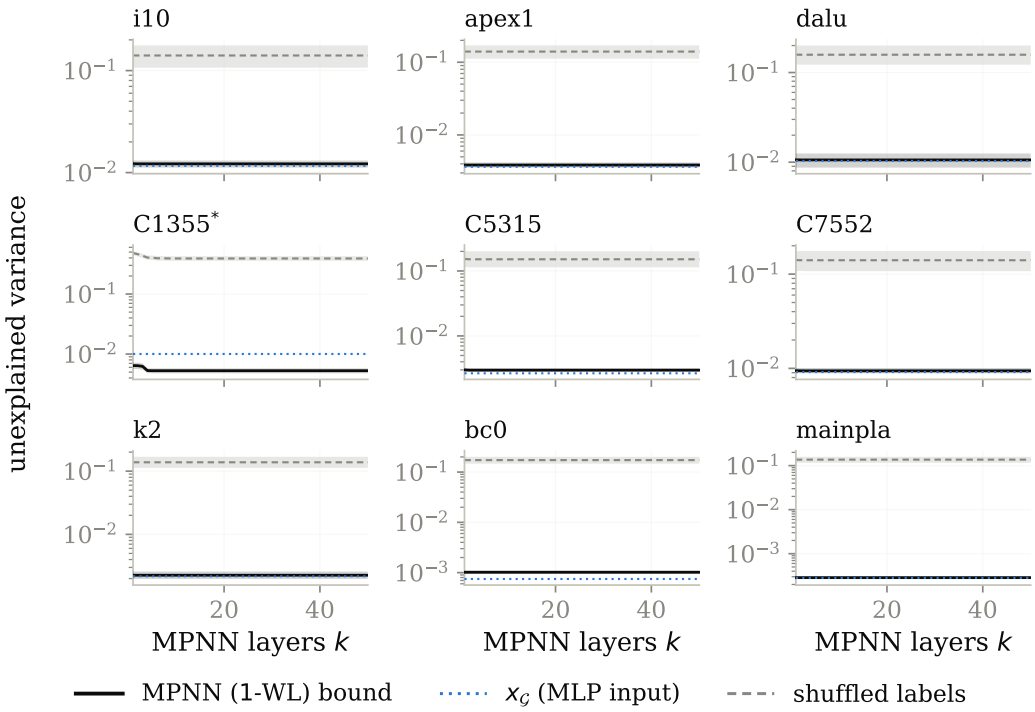

Figure 7: The MPNN bound as a function of depth k.

D Preliminary results: training models to predict $\widehat{V}$

The bounds of section 6.2 say what a representation *could* express. This appendix reports what models actually trained on the same datasets achieve. We report it as preliminary: the graph models do not fit their training set, and we have not established whether this comes from the models, from our training protocol, or from the task itself, so none of the claims of the paper rest on it.

Setup. We train eight regressors on the datasets of section 6.1. Four are generic message passing networks—GCN [17], GIN [45], GAT [41] and GraphSAGE [13]—taken from `PyTorch Geometric` [9]. Three are AIG-specific: DeepGate [18], PolarGate [21] and FuncGNN [46], each run from its authors' implementation. The eighth is the MLP baseline of section 4, which reads $x_{\mathcal{G}}$ and no graph at all, and is the baseline the graph models have to beat. All eight share the same head so that only the encoder differs: node embeddings are mean-pooled into one vector per AIG, which a two-layer ReLU network maps to a scalar. GCN, GIN and GraphSAGE consume the six-way one-hot node type of section 4; GAT, PolarGate and FuncGNN consume the three-way type (input, AND, inverter) of the DeepGate reader, with the inverter bit exposed as a signed edge feature, so that the models that were designed to use complementation receive it. DeepGate is the one exception to end-to-end training: we load the pretrained encoder, freeze it, and train only the head on its mean-pooled functional embeddings, which is how it is meant to be used as a general-purpose circuit encoder.

Targets are $\widehat{V}$ divided by the AND count of the benchmark's initial AIG, so they lie in $(0, 1]$ and are comparable across circuits. Each dataset of $1{,}000$ states is split $80/10/10$ into training, validation and test sets by a permutation seeded by the run seed. Models are trained for $1{,}000$ epochs with Adam [16] on the mean squared error. Each architecture is given its own grid of 36 configurations, and 12 for DeepGate, whose frozen encoder leaves only the head to tune; the grids are in appendix C. The grid is run at seed 0, the configuration with the lowest final validation error is selected, and that configuration alone is then rerun on the ten seeds we report; the seed selects the dataset, the split and the initialization together, so the ten repetitions are independent in all three. We report test error normalized by the variance of the test targets, so that 1.0 is the error of predicting the mean.

Results. All seven graph models land between 0.90 and 1.74 times the target variance—at or above the constant predictor—while the MLP reaches 0.47 at the median. Their training error is also ≈ 1, so this is a failure to fit rather than to generalize, and it leaves them two to three orders of magnitude above the bound of figure 3 that their own architecture class admits. The per-dataset validation curves of figure 9 show the graph models settling at the level of the constant predictor early in training and staying there. A gap of that size between what the class can express and what training reaches is what makes us treat these runs as preliminary rather than as a result about the architectures.

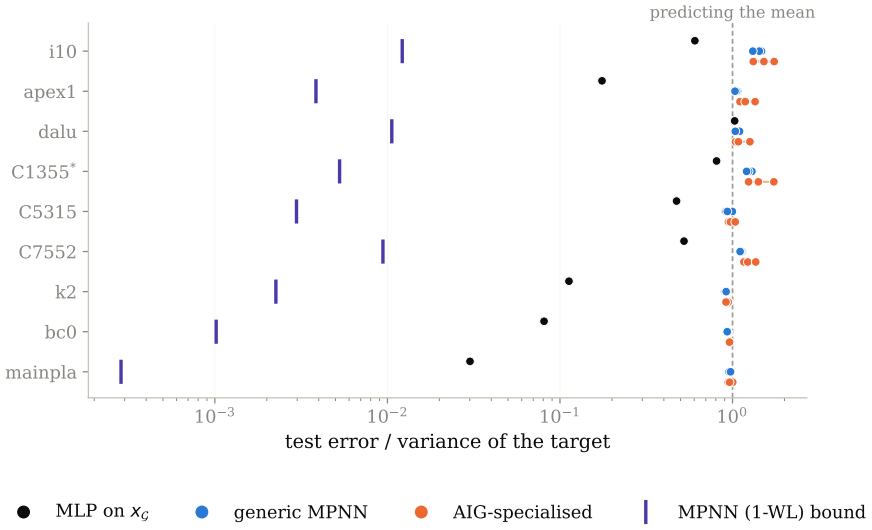

Figure 8: Test error of eight trained models, normalized by the variance of the target so that 1.0 is the error of predicting the mean. Violet ticks: the 1-WL bound of figure 3.

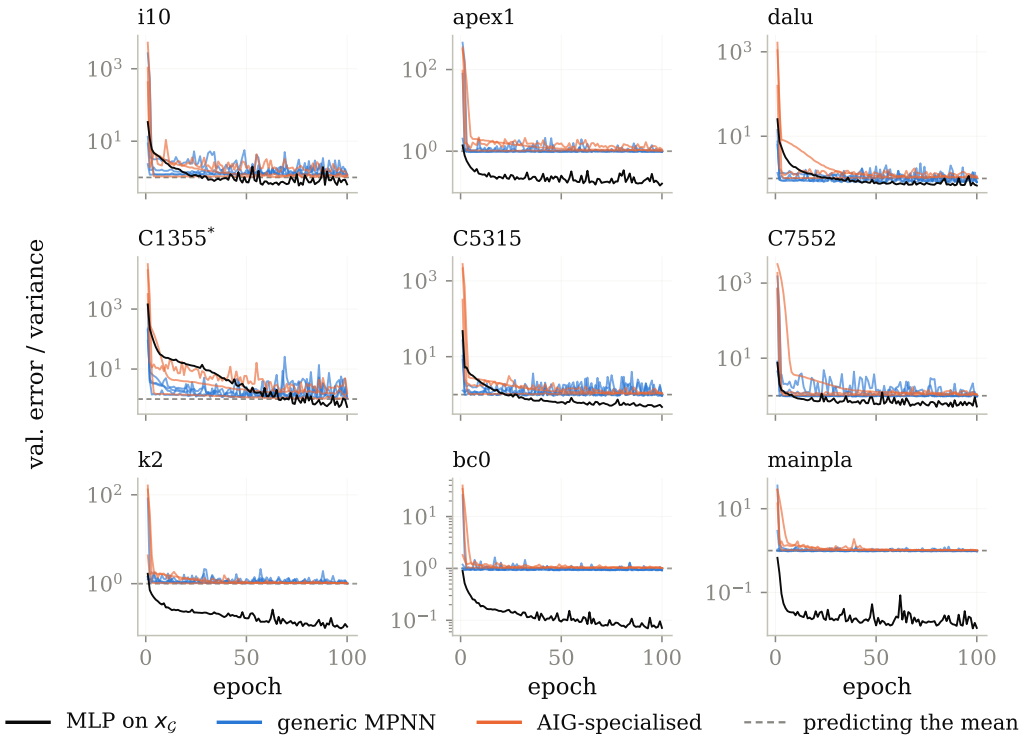

Figure 9: Validation error over training for the configurations of figure 8, one panel per benchmark.